

AI VIETNAM – AIO COURSE 2025

Hướng Dẫn Xây Dựng

Ứng Dụng Web Phân Tích Điểm Số Học Sinh Đơn Giản với Streamlit

Quân-Bảo Nguyễn, Mai-Anh Vũ, Quang-Vinh Đinh

23 Tháng 5 Năm 2025

1 Giới Thiệu Chung Về Streamlit

1.1 Streamlit là gì?



Hình 1: Streamlit

Streamlit là một thư viện mã nguồn mở của Python, được phát triển nhằm hỗ trợ việc xây dựng các ứng dụng web tương tác phục vụ cho khoa học dữ liệu và machine learning một cách nhanh chóng và thuận tiện. Framework này cho phép các lập trình viên chuyển đổi mã Python thông thường thành ứng dụng web chỉ với vài dòng lệnh đơn giản.

Với Streamlit, bạn có thể dễ dàng tạo dashboard, trực quan hóa dữ liệu qua các biểu đồ, cũng như triển khai các mô hình học máy, tất cả đều không yêu cầu kiến thức nền tảng về lập trình web.

1.2 Lợi ích của Streamlit

Streamlit là một lựa chọn phù hợp cho các ứng dụng web đơn giản, đặc biệt trong các dự án nhỏ hoặc dùng để trình diễn mô hình. Một số lợi ích chính gồm:

- Phù hợp với các dự án quy mô nhỏ, ứng dụng nội bộ hoặc demo nhanh.
- Không yêu cầu cấu hình bảo mật phức tạp.
- Thiết kế theo hướng một trang duy nhất, không cần liên kết nhiều trang.
- Giao diện đơn giản, dễ sử dụng, ít cần tùy chỉnh về layout.
- Có thể tích hợp với Flask, Django hoặc các công cụ frontend khác khi cần mở rộng.

2 Một số hàm thông dụng trong Streamlit

Các thành phần giao diện cơ bản trong Streamlit:

Các thành phần giao diện trong Streamlit là các yếu tố trực quan và tương tác, được sử dụng để xây dựng ứng dụng web. Mỗi thành phần cung cấp một chức năng cụ thể và có thể được tùy chỉnh linh hoạt để phù hợp với yêu cầu của ứng dụng. Dưới đây là một số thành phần cơ bản thường được sử dụng:

- `st.title("...")`: Dùng để tạo tiêu đề chính cho ứng dụng.
- `st.file_uploader("...", type=["..."])`: Tạo công cụ tải tệp lên ứng dụng. Tham số `type` cho phép giới hạn định dạng tệp được chấp nhận, ví dụ `["csv", "txt", "pdf"]`.
- `st.write("...")`: Dùng để hiển thị văn bản, bảng dữ liệu, biểu đồ hoặc các loại dữ liệu khác.
- `st.sidebar`: Tạo thanh điều hướng bên trái, giúp tổ chức các widget và điều khiển hợp lý hơn.
- `st.button("...")`: Tạo nút bấm cho phép thực thi hành động khi người dùng nhấn vào.
- `st.plotly_chart()`: Tạo biểu đồ tương tác sử dụng thư viện Plotly, hỗ trợ zoom, pan và các hành động tương tác khác.
- `st.line_chart()`, `st.bar_chart()`, `st.area_chart()`: Các hàm tích hợp sẵn để trực quan hóa dữ liệu dưới dạng biểu đồ đường, cột và vùng, thường sử dụng với `DataFrame` hoặc mảng `NumPy`.
- `st.map()`: Hiển thị dữ liệu địa lý trên bản đồ tương tác, áp dụng cho các tập dữ liệu có tọa độ kinh độ và vĩ độ.

Ngoài các thành phần trên, bạn có thể tham khảo thêm các widget và tính năng khác trong tài liệu chính thức của Streamlit để xây dựng các ứng dụng phong phú và tương tác hơn.

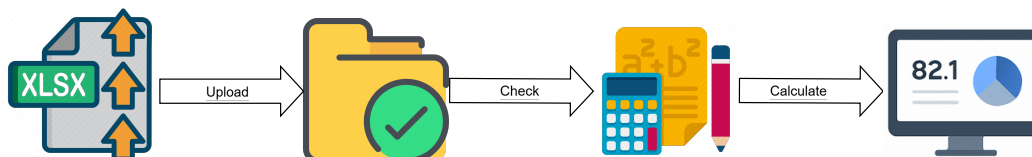
3 Thực Hành Xây Dựng Ứng Dụng Phân Tích Điểm Số Học Sinh

3.1 Giới thiệu ứng dụng

Trong phần thực hành này, ta sẽ xây dựng một ứng dụng hỗ trợ phân tích dữ liệu điểm số học sinh từ các file Excel. Các chức năng chính bao gồm: tính điểm trung bình, phân loại điểm theo từng khoảng, và trực quan hóa kết quả dưới dạng biểu đồ.

3.2 Pipeline tổng quan

Pipeline tổng quan của chương trình như sau: Người dùng upload file, hệ thống tự động tính toán và trả về kết quả phân tích cùng với biểu đồ trực quan.



Hình 2: Pipeline minh họa quá trình xử lý điểm số

3.3 Cài đặt môi trường và các thư viện cần thiết

Để xây dựng ứng dụng Streamlit một cách dễ dàng và tránh xung đột thư viện, ta nên tạo một môi trường ảo Python mới. Dưới đây là hai cách phổ biến để thực hiện: dùng `conda` hoặc `venv`.

Cách 1: Dùng conda (nếu bạn dùng Anaconda hoặc Miniconda)

1. Tạo môi trường mới:

```
conda create -n streamlit_app -y
```

- `-n streamlit_app`: chỉ định tên môi trường mới là `streamlit_app`.
- `-y`: tự động xác nhận (yes) tất cả các bước cài đặt, không cần hỏi lại người dùng.

2. Kích hoạt môi trường:

```
conda activate streamlit_app
```

Cách 2: Dùng venv (nếu không dùng Anaconda)

1. Tạo môi trường ảo:

```
python3 -m venv streamlit_env
```

2. Kích hoạt môi trường:

- Trên macOS hoặc Linux:

```
source streamlit_env/bin/activate
```

3.4 Cài đặt thư viện cần thiết

Sau khi kích hoạt môi trường, cài đặt các thư viện sau:

```
pip install streamlit pandas matplotlib openpyxl
```

- `streamlit`: streamlit xây dựng giao diện web tương tác cho ứng dụng Python.
- `pandas`: xử lý dữ liệu dạng bảng (DataFrame).
- `matplotlib`: vẽ biểu đồ, trực quan hóa dữ liệu.
- `openpyxl`: hỗ trợ đọc và ghi file Excel định dạng `.xlsx`.

3.5 Thiết Kế Kiến Trúc Ứng Dụng

1. Import các thư viện cần thiết

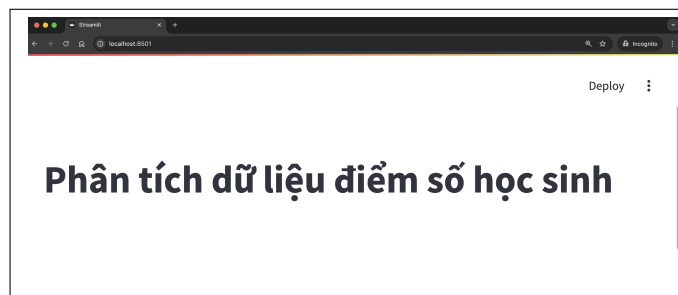
```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import streamlit as st
```

- streamlit: xây dựng giao diện web.
- pandas: đọc và xử lý dữ liệu từ file Excel.
- matplotlib.pyplot: vẽ biểu đồ (ở đây là biểu đồ tròn).

2. Tạo tiêu đề ứng dụng

```
1 st.title("Phân tích dữ liệu điểm số học sinh")
```

Sử dụng `st.title()` để tạo tiêu đề lớn trên giao diện web. Người dùng sẽ thấy dòng tiêu đề nổi bật khi mở ứng dụng.



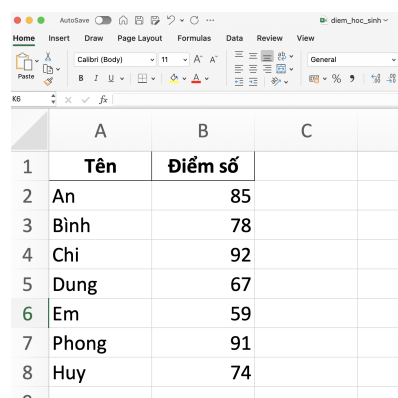
Hình 3: Hiển thị tiêu đề

3. Người dùng tải file lên

```
1 uploaded_file = st.file_uploader("Chọn file Excel (có cột 'Điểm số')",
  ↪ type=["xlsx"])
```

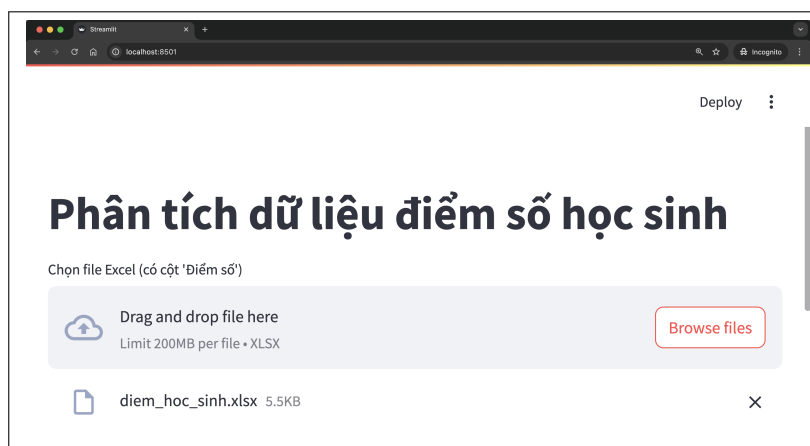
`st.file_uploader()` tạo nút chọn file để người dùng tải file Excel từ máy tính.

Tải lên file `.xlsx` chứa cột **Điểm số** như hình: **Lưu ý:** File phải đúng định dạng `.xlsx` và chứa cột tên "Điểm số".



	A	B	C
1	Tên	Điểm số	
2	An	85	
3	Bình	78	
4	Chi	92	
5	Dung	67	
6	Em	59	
7	Phong	91	
8	Huy	74	

(a) Chuẩn bị File Excel như hình



(b) Giao diện chương trình sau khi tải file Excel lên

Hình 4: Upload file Excel lên giao diện

Kiểm tra nếu người dùng đã upload file:

```
1 if uploaded_file:
```

Nếu người dùng đã tải file lên, ứng dụng sẽ tiếp tục xử lý các bước tiếp theo.

4. Viết các hàm xử lý dữ liệu

a. Hàm tính điểm trung bình

```
1 def calculate_average(scores):
2     return sum(scores) / len(scores)
```

Hàm cộng tổng tất cả điểm rồi chia cho số lượng học sinh.

b. Hàm phân loại điểm số

```
1 def percentage_distribution(scores):
2     bins = {'90-100': 0, '80-89': 0, '70-79': 0, '60-69': 0, '<60': 0}
3     for score in scores:
4         if score >= 90:
5             bins['90-100'] += 1
6         elif score >= 80:
7             bins['80-89'] += 1
8         elif score >= 70:
9             bins['70-79'] += 1
10        elif score >= 60:
11            bins['60-69'] += 1
12        else:
13            bins['<60'] += 1
14    return bins
```

Hàm chia điểm số thành 5 nhóm và đếm số lượng học sinh trong mỗi nhóm.

5. Đọc file Excel và hiển thị trung bình

```
1 df = pd.read_excel(uploaded_file)
```

Xử lý danh sách điểm số:

```
1 scores = df["Điểm số"].dropna().astype(float).tolist()
```

Hiển thị các chỉ số:

```
1 st.write("Tổng số học sinh:", len(scores))
2 st.write("Điểm trung bình:", round(calculate_average(scores), 2))
```

6. Vẽ biểu đồ

Ta chuyển kết quả phân loại về dạng danh sách như sau:

```
1 dist = percentage_distribution(scores)
2 labels = list(dist.keys())
3 values = list(dist.values())
```

Trong đó:

- **labels**: Danh sách tên của các nhóm điểm số (ví dụ: "90-100", "80-89", ...).
- **values**: Danh sách số lượng học sinh tương ứng với từng nhóm điểm số.

Vẽ biểu đồ tròn

```
1 fig, ax = plt.subplots(figsize=(3, 3))
2 ax.pie(values, labels=labels, autopct='%1.1f%%', startangle=90)
3 ax.axis("equal")
4 st.pyplot(fig)
```

- **plt.subplots(figsize=(1, 1))**: Tạo biểu đồ với kích thước trung bình (1 inch × 1 inch).
- **ax.pie(values, labels=labels, autopct='%1.1f%', startangle=90)**: Vẽ biểu đồ tròn với:
 - **values**: Giá trị tương ứng với từng phần của biểu đồ.
 - **labels**: Tên nhóm tương ứng với từng phần của biểu đồ.
 - **autopct='%1.1f%'**: **autopct** là viết tắt của "automatic percentage", dùng để tự động hiển thị tỷ lệ phần trăm trên từng phần của biểu đồ. Định dạng '%1.1f%' chỉ định rằng phần trăm sẽ được hiển thị dưới dạng số thực với một chữ số sau dấu thập phân.
- **ax.axis("equal")**: Đảm bảo biểu đồ cân đối (tỉ lệ trục X và Y bằng nhau).
- **st.pyplot(fig)**: Hiển thị biểu đồ trên ứng dụng Streamlit.

Hiển thị biểu đồ với độ phân giải cao

Cách 1: Vẽ trực tiếp bằng st.pyplot(fig)

```

1 st.pyplot(fig, use_container_width=False)
2 st.markdown("Biểu đồ phân bố điểm số.")

```

Cách này độ nét của hình ảnh phụ thuộc vào thông số `figsize` và độ phân giải mặc định. Điều này có thể dẫn đến hiện tượng hình ảnh bị mờ khi hiển thị trên giao diện Streamlit, như minh họa tại Hình 5a.

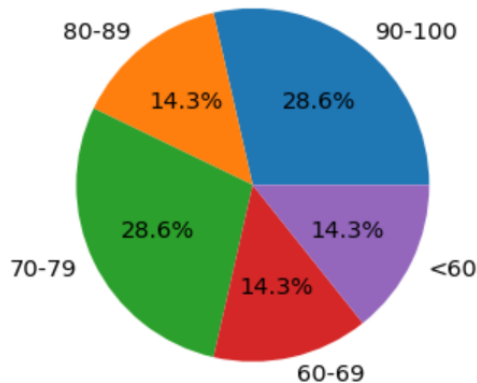
Cách 2: Lưu biểu đồ thành ảnh rồi hiển thị.

Phương pháp này lưu biểu đồ dưới định dạng PNG với độ phân giải cao (`dpi=300` trước khi hiển thị bằng `st.image`). Điều này giúp đảm bảo hình ảnh rõ nét, đồng thời cho phép kiểm soát chính xác kích thước khi hiển thị (ví dụ, thiết lập `width=250px`). Do đó, cách 2 được khuyến khích sử dụng trong các ứng dụng yêu cầu chất lượng trình bày cao, như minh họa tại Hình 5b.

```

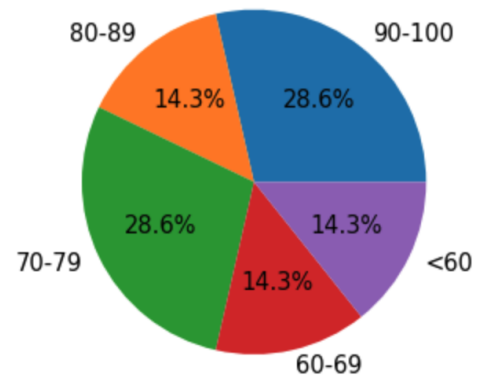
1 import io
2 from PIL import Image
3
4 buf = io.BytesIO()
5 fig.savefig(buf, format='png', dpi=300) # Lưu với dpi cao để ảnh sắc nét
6 buf.seek(0)
7
8 st.markdown("Biểu đồ phân bố điểm số.")
9 img = Image.open(buf)
10 st.image(img, width=250) # Hiển thị ảnh với kích thước cố định

```



Biểu đồ phân bố điểm số.

(a) Cách 1



Biểu đồ phân bố điểm số.

(b) Cách 2 (khuyến khích sử dụng)

Hình 5: So sánh hình ảnh trước và sau xử lý.

Thêm ghi chú nhỏ dưới biểu đồ

```
1 st.markdown("Biểu đồ phân bố điểm số.")
```

Đưa biểu đồ vào giữa giao diện:

Để biểu đồ hiển thị ở vị trí trung tâm thay vì căn trái mặc định, ta có thể sử dụng chức năng chia cột trong Streamlit.

Cụ thể, ta tạo ba cột với tỷ lệ [1, 2, 1], trong đó cột giữa có độ rộng lớn hơn để chứa biểu đồ. Sau đó, sử dụng `with col2:` để đặt biểu đồ vào cột giữa, đảm bảo nội dung được căn giữa giao diện.

```
1 col1, col2, col3 = st.columns([1, 2, 1]) # Tạo ba cột, cột giữa rộng hơn
2 with col2:
3     st.image(img, width=300) # Hiển thị biểu đồ ở cột giữa
4     st.markdown("Biểu đồ phân bố điểm số.")
```

Hoàn thiện ứng dụng:

Sau khi hoàn thành các bước trên, ta thu được một tập tin `app.py` hoàn chỉnh, sẵn sàng để chạy trực tiếp trên nền tảng Streamlit.

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3 import streamlit as st
4 import io
5 from PIL import Image
6
7
8 # Tiêu đề ứng dụng
9 st.title("Phân tích dữ liệu điểm số học sinh")
10
11 # Upload file
12 uploaded_file = st.file_uploader("Chọn file Excel (có cột 'Điểm số')",
13     ↪ type=["xlsx"])
14
15 # Hàm tính điểm trung bình
16 def calculate_average(scores):
17     return sum(scores) / len(scores)
18
19 # Hàm phân loại điểm số
20 def percentage_distribution(scores):
21     bins = {'90-100': 0, '80-89': 0, '70-79': 0, '60-69': 0, '<60': 0}
22     for score in scores:
23         if score >= 90:
24             bins['90-100'] += 1
25         elif score >= 80:
26             bins['80-89'] += 1
27         elif score >= 70:
28             bins['70-79'] += 1
29         elif score >= 60:
30             bins['60-69'] += 1
```

```

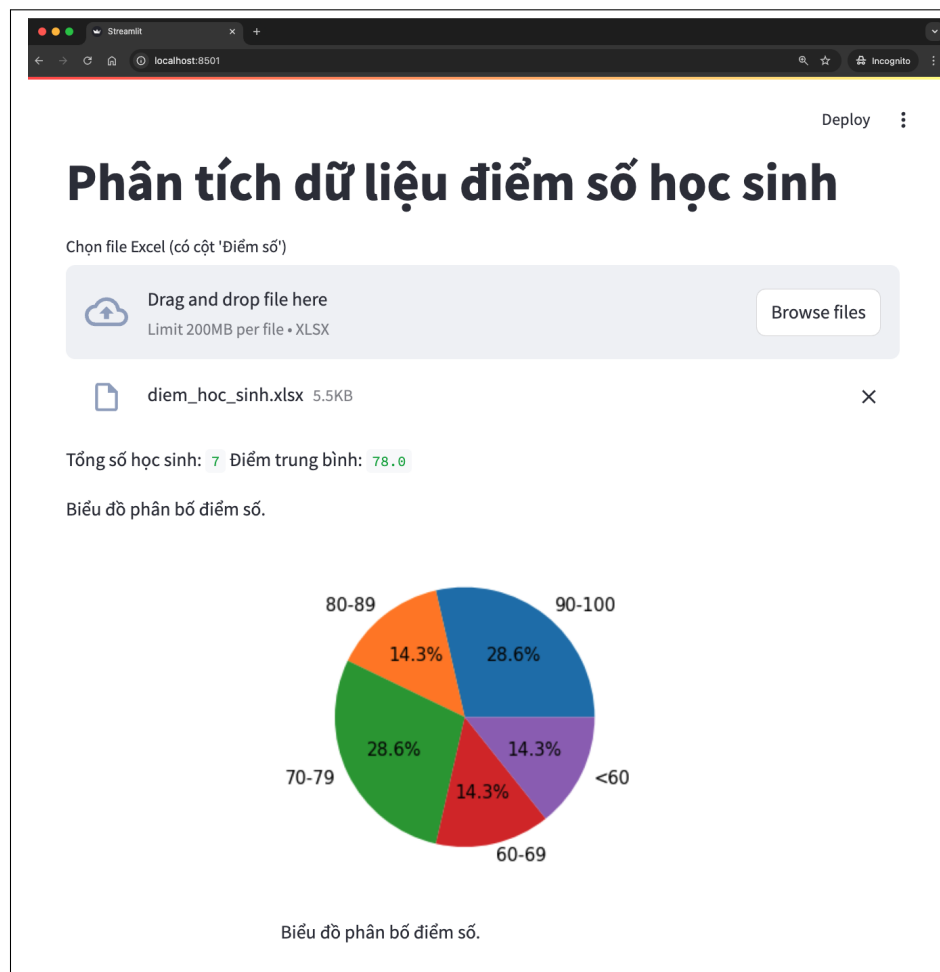
30         else:
31             bins['<60'] += 1
32         return bins
33
34     # Khi có file
35     if uploaded_file:
36         df = pd.read_excel(uploaded_file)
37         scores = df["Điểm số"].dropna().astype(float).tolist()
38
39         if scores:
40             st.write("Tổng số học sinh:", len(scores), "Điểm trung bình:",
41                     ↪ round(calculate_average(scores), 2))
42             # Phân loại điểm
43             dist = percentage_distribution(scores)
44             labels = list(dist.keys())
45             values = list(dist.values())
46
47             fig, ax = plt.subplots(figsize=(1, 1))
48             ax.pie(
49                 values,
50                 labels=labels,
51                 autopct='%1.1f%%',
52                 textprops={'fontsize': 3.5},
53             )
54             ax.axis("equal")
55             plt.tight_layout(pad=0.1)
56
57             buf = io.BytesIO()
58             fig.savefig(buf, format='png', dpi=300)
59             buf.seek(0)
60             st.markdown("Biểu đồ phân bố điểm số.")
61             img = Image.open(buf)
62
63             col1, col2, col3 = st.columns([1, 2, 1]) # Tạo ba cột, cột giữa rộng hơn
64             with col2:
65                 st.image(img, width=300) # Hiển thị biểu đồ ở cột giữa
66                 st.markdown("Biểu đồ phân bố điểm số.")

```

Sau khi viết xong code (ví dụ lưu thành file `app.py`), bạn chạy ứng dụng bằng

```
streamlit run app.py
```

Sau khi hoàn thành các bước xây dựng, giao diện người dùng (UI) của ứng dụng sẽ hiển thị như sau:



Hình 6: Giao diện ứng dụng Streamlit sau khi triển khai.

- Hết -